

JavaScript Arrays

Table of Contents

1. [What is an Array?](#)
 2. [Array Terminology](#)
 3. [Ways to Create Arrays](#)
 4. [Array Methods](#)
 - [Mutating Methods](#)
 - [Non-Mutating Methods](#)
 - [Higher-Order Methods](#)
 - [Search Methods](#)
 - [Testing Methods](#)
-

What is an Array?

Key Characteristics

- **Array is an object** in JavaScript
- **Non-primitive data type** (reference type)
- **Block of memory** used to store multiple values of different types
- Elements are **ordered** and **indexed** starting from 0

Example

```
const arr = [12, 10n, true, 'abc', null, undefined];
console.log(typeof arr); // "object"
```

Array Terminology

Term	Definition	Example
Length	Total number of elements in array	arr.length
Index	Integer number starting from 0	arr[0], arr[1], arr[2]
Array Elements	Values stored inside square brackets	[12, 'hello', true]
Empty Array	Array with no elements	[]

Visual Representation

```
Index:    0      1      2      3
Array: [ 12,  10n,  true, 'abc' ]
Length: 4
```

Ways to Create Arrays

1. Array Literal (Square Brackets)

```
const arr1 = [10, 20, 30];
const arr2 = []; // Empty array
const mixed = [1, 'hello', true, null];
```

2. Array Constructor (new keyword)

```
// With elements
const arr3 = new Array('html', 'css', 'js');

// Empty array with specific length
const arr4 = new Array(5); // Creates array with 5 empty slots

// Single element (be careful!)
const arr5 = new Array(10); // Creates array with length 10, not [10]
const arr6 = new Array('10'); // Creates ['10']
```

Array Constructor Details

- **Built-in constructors** always start with uppercase letters
- **new** keyword creates an empty object and shares reference via **this**
- **Array()** is a built-in constructor function that loads and initializes values

```
const skills = new Array('html', 'css', 'js');

// Internal structure:
// Index:    0      1      2
// Value:    'html' 'css' 'js'
// Length:   3
// Prototype: Array[]
```

Array Methods

Mutating Methods (Modify Original Array)

1. push()* - Add to End

Adds one or more elements to the end of array and returns new length.

```
const fruits = ['apple', 'banana'];
const newLength = fruits.push('orange', 'mango');

console.log(fruits);    // ['apple', 'banana', 'orange', 'mango']
console.log(newLength); // 4
```

2. pop()* - Remove from End

Removes and returns the last element from array.

```
const fruits = ['apple', 'banana', 'orange'];
const removed = fruits.pop();

console.log(fruits); // ['apple', 'banana']
console.log(removed); // 'orange'
```

3. unshift()* - Add to Beginning

Adds one or more elements to the beginning of array and returns new length.

```
const numbers = [2, 3, 4];
const newLength = numbers.unshift(0, 1);

console.log(numbers);    // [0, 1, 2, 3, 4]
console.log(newLength);  // 5
```

4. shift()* - Remove from Beginning

Removes and returns the first element from array.

```
const numbers = [0, 1, 2, 3];
const removed = numbers.shift();

console.log(numbers); // [1, 2, 3]
console.log(removed); // 0
```

5. splice()* - Insert, Delete, Update

Most versatile method for array manipulation.

Syntax: `array.splice(start, deleteCount, item1, item2, ...)`

```
const arr = ['a', 'b', 'c', 'd', 'e'];

// Delete 2 elements starting from index 1
const deleted = arr.splice(1, 2);
console.log(arr);    // ['a', 'd', 'e']
console.log(deleted); // ['b', 'c']

// Insert elements at index 1
arr.splice(1, 0, 'x', 'y');
console.log(arr); // ['a', 'x', 'y', 'd', 'e']

// Replace elements
arr.splice(1, 2, 'NEW');
console.log(arr); // ['a', 'NEW', 'd', 'e']
```

6. reverse() - Reverse Array

Reverses array in place and returns reference to same array.

```
const numbers = [1, 2, 3, 4, 5];
const reversed = numbers.reverse();

console.log(numbers); // [5, 4, 3, 2, 1] - original modified
console.log(reversed); // [5, 4, 3, 2, 1] - same reference
console.log(numbers === reversed); // true
```

7. sort()* - Sort Array

Sorts elements in place and returns sorted array.

```
const numbers = [3, 1, 4, 1, 5, 9, 2];

// Default: converts to strings and sorts alphabetically
numbers.sort();
console.log(numbers); // [1, 1, 2, 3, 4, 5, 9]

// Numeric sorting
const nums = [10, 5, 20, 1];

// Ascending order
nums.sort((a, b) => a - b);
console.log(nums); // [1, 5, 10, 20]

// Descending order
nums.sort((a, b) => b - a);
console.log(nums); // [20, 10, 5, 1]
```

Non-Mutating Methods (Don't Modify Original)

8. slice()* - Extract Portion

Returns shallow copy of portion of array.

Syntax: `array.slice(start?, end?)`

```
const arr = ["html", "css", "js", "java", "sql", "dsa"];

console.log(arr.slice());           // ['html', 'css', 'js', 'java', 'sql', 'dsa']
- full copy
console.log(arr.slice(2));          // ['js', 'java', 'sql', 'dsa'] - from index 2
console.log(arr.slice(1, 4));       // ['css', 'js', 'java'] - from index 1 to 3
console.log(arr.slice(-2));         // ['sql', 'dsa'] - last 2 elements
console.log(arr.slice(-4, -1));     // ['js', 'java', 'sql'] - negative indices

console.log(arr); // Original unchanged: ['html', 'css', 'js', 'java', 'sql', 'dsa']
```

9. includes()* - Check Element Exists

Returns true if element exists, false otherwise.

```
const fruits = ['apple', 'banana', 'orange'];

console.log(fruits.includes('banana')); // true
console.log(fruits.includes('grape'));  // false
console.log(fruits.includes('Apple'));  // false - case sensitive

// With start index
console.log(fruits.includes('apple', 1)); // false - start search from index 1
```

10. indexOf() - Find Element Index

Returns first index of element, or -1 if not found.

```
const numbers = [1, 2, 3, 2, 4];

console.log(numbers.indexOf(2));    // 1 - first occurrence
console.log(numbers.indexOf(5));    // -1 - not found
console.log(numbers.indexOf(2, 2)); // 3 - start search from index 2
```

Higher-Order Methods (Accept Callback Functions)

11. forEach()* - Iterate Elements

Executes function for each element. **Always returns undefined.**

```
const nums = [10, 20, 30, 40];

// Basic usage
nums.forEach(function task(element, index, array) {
  console.log(`Element at ${index}:`, element);
});
// Output:
// Element at 0: 10
// Element at 1: 20
// Element at 2: 30
// Element at 3: 40

// Even if you return something, forEach ignores it
const result = nums.forEach(function(element) {
  return 'hello'; // This return value is ignored
});
console.log(result); // undefined - forEach always returns undefined
```

Key Points about forEach:

- **Higher-order function** (accepts callback)
- **Cannot be stopped** in middle (no break/continue)
- **Not visible in call stack** (built-in method)
- **Syntax:** `forEach(callback, thisArg)`

12. map()* - Transform Elements

Creates new array with results of calling function on every element.

```
const numbers = [1, 2, 3, 4, 5];

const squared = numbers.map((num) => num ** 2);
console.log(squared); // [1, 4, 9, 16, 25]
console.log(numbers); // [1, 2, 3, 4, 5] - original unchanged
```

13. filter()* - Select Elements

Creates new array with elements that pass test function.


```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

const evenNumbers = numbers.filter((num) => num % 2 === 0);
console.log(evenNumbers); // [2, 4, 6, 8, 10]

const greaterThanFive = numbers.filter((num) => num > 5);
console.log(greaterThanFive); // [6, 7, 8, 9, 10]
```

14. reduce()* - Accumulate Values

Reduces array to single value by executing reducer function.

```
const numbers = [1, 2, 3, 4, 5];

// Sum all numbers
const sum = numbers.reduce((accumulator, current) => {
  return accumulator + current;
}, 0);
console.log(sum); // 15

// Find maximum
const max = numbers.reduce((acc, current) => {
  return current > acc ? current : acc;
}, numbers[0]);
console.log(max); // 5
```

15. reduceRight() - Reduce from Right

Same as reduce but processes from right to left.

```
const numbers = [1, 2, 3, 4, 5];

const result = numbers.reduceRight((acc, current, index) => {
  console.log(`Index ${index}: ${current}`);
  return acc + current;
}, 0);

// Output:
// Index 4: 5
// Index 3: 4
// Index 2: 3
// Index 1: 2
// Index 0: 1
```

16. flat() - Flatten Nested Arrays

Creates new array with sub-arrays flattened.

```
const nested = [1, [2, 3], [4, [5, 6]], 7];

console.log(nested.flat());           // [1, 2, 3, 4, [5, 6], 7] - depth 1
console.log(nested.flat(2));          // [1, 2, 3, 4, 5, 6, 7] - depth 2
console.log(nested.flat(Infinity));   // [1, 2, 3, 4, 5, 6, 7] - all levels
```

17. flatMap() - Map then Flatten

Maps each element then flattens result by one level.

```
const sentences = ["Hello world", "How are you"];

const words = sentences.flatMap(sentence => sentence.split(' '));
console.log(words); // ['Hello', 'world', 'How', 'are', 'you']

// Equivalent to:
const wordsManual = sentences.map(sentence => sentence.split(' ')).flat();
```

Search Methods

19. find()* - Find First Element

Returns first element that satisfies test function.

```
const users = [  
  { id: 1, name: 'Alice', age: 25 },  
  { id: 2, name: 'Bob', age: 30 },  
  { id: 3, name: 'Charlie', age: 35 }  
];  
  
const user = users.find(user => user.age > 28);  
console.log(user); // { id: 2, name: 'Bob', age: 30 }  
  
const notFound = users.find(user => user.age > 40);  
console.log(notFound); // undefined
```

20. findLast() - Find Last Element

Returns last element that satisfies test function.

```
const numbers = [1, 2, 3, 4, 5, 4, 3, 2, 1];  
  
const lastEven = numbers.findLast(num => num % 2 === 0);  
console.log(lastEven); // 2 (last even number)
```

21. findIndex() - Find First Index

Returns index of first element that satisfies test function.

```
const numbers = [5, 12, 8, 130, 44];  
  
const index = numbers.findIndex(num => num > 10);  
console.log(index); // 1 (index of 12)  
  
const notFoundIndex = numbers.findIndex(num => num > 200);  
console.log(notFoundIndex); // -1
```

22. findLastIndex() - Find Last Index

Returns index of last element that satisfies test function.

```
const numbers = [5, 12, 8, 130, 44];

const lastIndex = numbers.findLastIndex(num => num > 10);
console.log(lastIndex); // 4 (index of 44)
```

Testing Methods

23. some()* - Test Some Elements

Returns true if at least one element passes test.

```
const numbers = [1, 2, 3, 4, 5];

const hasEven = numbers.some(num => num % 2 === 0);
console.log(hasEven); // true (2 and 4 are even)

const hasNegative = numbers.some(num => num < 0);
console.log(hasNegative); // false

// Real-world example
const users = [
  { name: 'Alice', age: 17 },
  { name: 'Bob', age: 25 },
  { name: 'Charlie', age: 16 }
];

const hasAdult = users.some(user => user.age >= 18);
console.log(hasAdult); // true (Bob is 25)
```

24. every()* - Test All Elements

Returns true if all elements pass test.

```
const numbers = [2, 4, 6, 8, 10];

const allEven = numbers.every(num => num % 2 === 0);
console.log(allEven); // true (all are even)

const allPositive = numbers.every(num => num > 0);
console.log(allPositive); // true (all are positive)

const allGreaterThanFive = numbers.every(num => num > 5);
console.log(allGreaterThanFive); // false (2 and 4 are not > 5)

// Real-world example
const products = [
  { name: 'Laptop', price: 1000, inStock: true },
  { name: 'Phone', price: 500, inStock: true },
  { name: 'Tablet', price: 300, inStock: false }
];

const allInStock = products.every(product => product.inStock);
console.log(allInStock); // false (Tablet is out of stock)
```

Method Categories Summary

Mutating Methods (Change Original Array)

- `push()`, `pop()`, `unshift()`, `shift()`
- `splice()`, `reverse()`, `sort()`

Non-Mutating Methods (Return New Array/Value)

- `slice()`, `includes()`, `indexOf()`
- All higher-order methods: `map()`, `filter()`, `reduce()`, etc.

Performance Considerations

Method Type	Time Complexity	Best Use Case
push/pop	O(1)	Add/remove from end

Method Type	Time Complexity	Best Use Case
unshift/shift	O(n)	Add/remove from beginning
splice	O(n)	Insert/delete at any position
slice	O(n)	Copy array portion
Higher-order	O(n)	Transform/filter/search

Common Patterns and Best Practices

1. Array Copying

```
// Shallow copy methods
const arr = [1, 2, 3, 4, 5];

const copy1 = [...arr];           // Spread operator
const copy2 = arr.slice();        // slice method
const copy3 = Array.from(arr);    // Array.from method
```

2. Array Checking

```
// Check if variable is array
const arr = [1, 2, 3];

console.log(Array.isArray(arr));    // true
console.log(arr instanceof Array);  // true
console.log(Object.prototype.toString.call(arr)); // '[object Array]'
```

3. Method Chaining

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

const result = numbers
  .filter(num => num % 2 === 0)    // [2, 4, 6, 8, 10]
  .map(num => num * 2)            // [4, 8, 12, 16, 20]
  .reduce((sum, num) => sum + num, 0); // 60

console.log(result); // 60
```

4. Empty Array Check

```
const arr = [];

// Different ways to check if array is empty
console.log(arr.length === 0);    // true
console.log(arr.length);          // 0
console.log(!arr.length);         // true
```

Quick Reference Table

Method	Mutates	Returns	Use Case
push()	✓	New length	Add to end
pop()	✓	Removed element	Remove from end
unshift()	✓	New length	Add to start
shift()	✓	Removed element	Remove from start
splice()	✓	Deleted elements array	Insert/delete/replace
slice()	✗	New array	Copy portion
includes()	✗	Boolean	Check existence
sort()	✓	Sorted array	Sort elements
reverse()	✓	Reversed array	Reverse order

Method	Mutates	Returns	Use Case
<code>forEach()</code>	✗	undefined	Iterate
<code>map()</code>	✗	New array	Transform
<code>filter()</code>	✗	New array	Select
<code>reduce()</code>	✗	Single value	Accumulate
<code>find()</code>	✗	Element or undefined	Find first
<code>some()</code>	✗	Boolean	Test any
<code>every()</code>	✗	Boolean	Test all

This comprehensive guide covers all JavaScript array fundamentals and methods with practical examples and best practices for effective array manipulation.